

ESILV / Big Data Module

Final Report

Big Data Crypto Streaming Project

End-to-end pipeline: **Kafka** → **Spark** → **InfluxDB** → **Grafana**

Student: Anas Khalil & Mourad Mahmoudi

Program: M2 Computer Science & Data Science

Date: 18 January 2026

GitHub repository:

<https://github.com/Anas-kh3/bigdata-crypto-streaming>

Keywords: Big Data, Streaming, Kafka, Spark Structured Streaming, HDFS, InfluxDB, Grafana, Time-Series Analytics

Contents

1	Introduction	2
2	Global Architecture	2
3	Infrastructure and Environment	2
4	Data Ingestion — Kafka	3
5	Raw Data Storage — HDFS	3
6	Data Processing — Spark	3
7	Time-Series Storage — InfluxDB	4
8	Visualization — Grafana	4
9	Results and Analysis	4
10	Limitations and Improvements	4
10.1	Current Limitations	4
10.2	Future Improvements	5
11	Conclusion	5

1 Introduction

This project is carried out as part of the **Big Data** module. The objective is to design and implement a complete end-to-end streaming pipeline capable of ingesting, processing, storing, and visualizing near real-time cryptocurrency market data.

The project focuses on **Bitcoin (BTC)** and **Ethereum (ETH)**, two highly liquid assets, which makes them suitable for real-time monitoring and time-series analytics.

This final report consolidates the work delivered during *Checkpoint 1*, *Checkpoint 2*, and the final implementation phase. All code, scripts, and documentation are available on GitHub: <https://github.com/Anas-kh3/bigdata-crypto-streaming>.

2 Global Architecture

The system follows a classical streaming architecture organized into clear layers:

- **Ingestion Layer:** Kafka collects real-time data from external APIs.
- **Processing Layer:** Spark Structured Streaming performs transformations and aggregations.
- **Storage Layer:** HDFS stores raw events, and InfluxDB stores time-series metrics.
- **Visualization Layer:** Grafana dashboards display real-time analytics.

Data flows continuously from external APIs to Kafka, then to Spark, and finally to InfluxDB, where dashboards are built using Grafana.

3 Infrastructure and Environment

The project is deployed on a cluster composed of **four virtual machines**:

- **2 master nodes**
- **2 worker nodes**

Each node runs Linux and the cluster is configured with:

- Apache Kafka (real-time ingestion)
- Apache Hadoop (HDFS storage)
- Apache Spark (stream processing)
- InfluxDB v2 (time-series database)
- Grafana (visualization)

All services communicate over a private network, ensuring stable intra-cluster connectivity and performance.

4 Data Ingestion — Kafka

Data is ingested from two public sources:

- **CoinGecko API** (price + 24h volume)
- **Binance Public API** (price + 24h quote volume)

Python producers poll the APIs every few seconds and publish standardized JSON messages to Kafka.

Each message contains:

- **symbol** (btc, eth)
- **price** (USD)
- **volume_24h** (USD)
- **timestamp** (event time)
- **source** (coingecko, binance)

Kafka provides buffering, scalability, and decoupling between ingestion and processing.

5 Raw Data Storage — HDFS

Spark Structured Streaming consumes Kafka topics and writes raw events into HDFS. Data is partitioned by day, which allows:

- efficient batch reprocessing
- historical analysis
- reproducibility (raw immutable events)

This layer acts as a **data lake**.

6 Data Processing — Spark

Spark Structured Streaming performs real-time transformations such as:

- window-based aggregation (1-minute average price)
- computation of derived indicators used in dashboards
- preparation of curated time-series metrics

To ensure stable streaming behavior, watermarking and windowing are applied to handle late events and limit state growth.

7 Time-Series Storage — InfluxDB

Processed metrics are stored in InfluxDB (time-series oriented), chosen for:

- high write throughput
- efficient queries over time ranges
- native integration with Grafana

Two main measurements are stored:

- **crypto_1min:** windowed metrics (avg price, aggregated volume)
- **crypto_latest:** latest snapshot (price, volume_24h)

Tags: symbol, source. Fields: avg_price, price, volume_24h, sum_volume_24h.

8 Visualization — Grafana

Grafana dashboards provide real-time monitoring and comparison between BTC and ETH.

Implemented panels include:

- **BTC vs ETH price** (time series)
- **Latest BTC / ETH price** (stat)
- **Normalized Price (base = 100 at start)** — BTC & ETH
- **BTC/ETH ratio** (relative performance indicator)

These visualizations help detect trends, compare assets, and monitor relative performance within the selected time range.

9 Results and Analysis

From the produced dashboards, the following observations can be made:

- BTC and ETH show a strong correlation over short time windows.
- The normalized price view highlights relative movements independent of scale.
- The BTC/ETH ratio provides a direct view of which asset is outperforming the other.

Overall, the pipeline operates in near real-time and successfully delivers end-to-end analytics from ingestion to visualization.

10 Limitations and Improvements

10.1 Current Limitations

- Only two assets (BTC, ETH) are monitored.
- The project focuses on core metrics (price, volume) without advanced indicators.

10.2 Future Improvements

- Add alerting (e.g., volatility spikes, threshold crossing).
- Add technical indicators (moving average, RSI, volatility estimation).
- Production deployment using Docker and/or Kubernetes.
- Extend to more assets and more exchanges.

11 Conclusion

This project demonstrates the implementation of a full Big Data streaming architecture from ingestion to visualization.

Kafka ensures scalable ingestion, Spark handles streaming transformations, HDFS stores raw events for reproducibility, InfluxDB provides efficient time-series storage, and Grafana enables interactive monitoring.

The final system validates both theoretical Big Data concepts and practical data engineering skills through a complete end-to-end implementation.